CPE 486/586: Deep Learning for Engineering Applications

04 Training Neural Networks and Computational Thinking

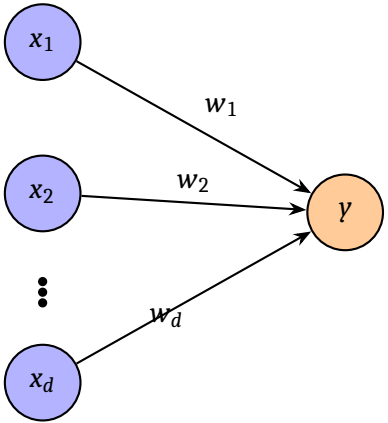
Spring 2026

Rahul Bhadani

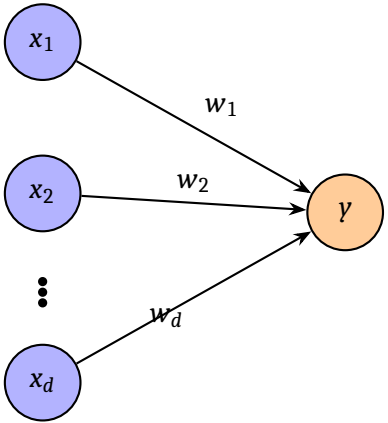
Outline

Recap

Linear Layer

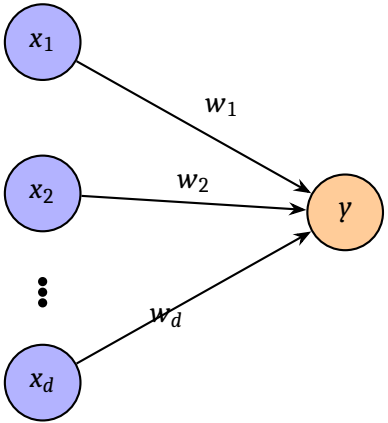


Linear Layer



$$y = \sum_{i=1}^{n} w_i x_i$$

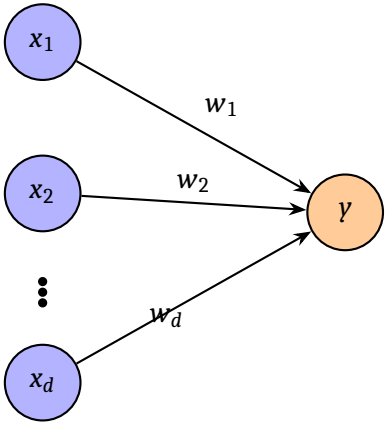
Linear Layer



$$y = \sum_{i=1}^{n} w_i x_i$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}$$

Linear Layer

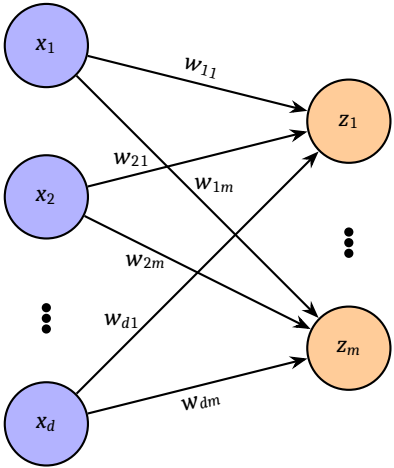


$$y = \sum_{i=1}^{n} w_i x_i$$

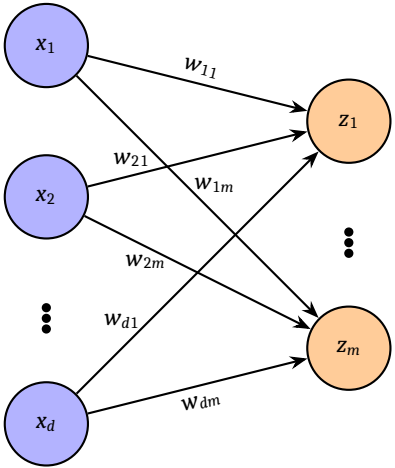
$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}$$

$$\Rightarrow y = \mathbf{x}\mathbf{w}$$

Linear Layer: Case of Hidden Layer

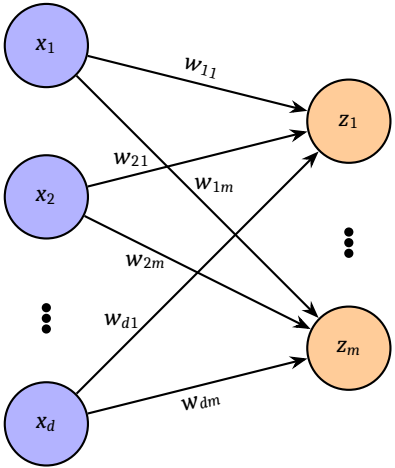


Linear Layer: Case of Hidden Layer



$$z_m = \sum_{i=1}^{n} x_i w_{im}$$

Linear Layer: Case of Hidden Layer

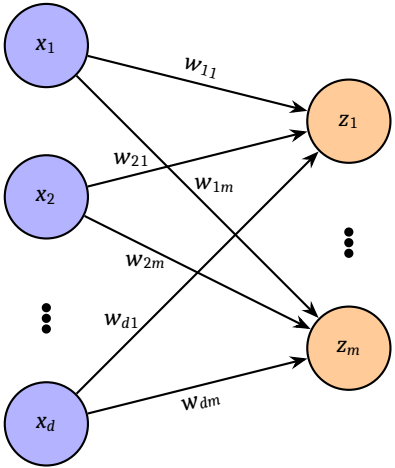


$$z_m = \sum_{i=1}^{n} x_i w_{im}$$

$$\mathbf{z} = \begin{bmatrix} z_1 & z_2 & \vdots & z_m \end{bmatrix}_{1 \times m}$$

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1m} \\ w_{21} & w_{22} & \cdots & w_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ w_{d1} & w_{m2} & \cdots & w_{dm} \end{bmatrix}_{d \times m} \qquad \mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}_{1 \times d}$$

Linear Layer: Case of Hidden Layer



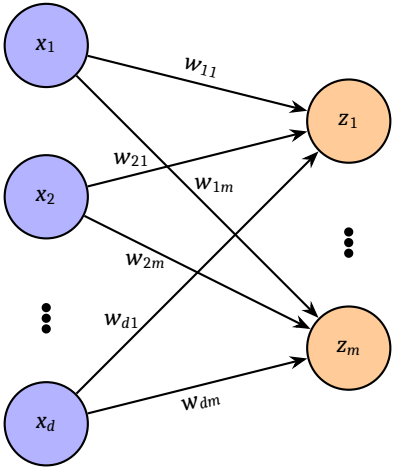
$$z_m = \sum_{i=1}^{n} x_i w_{im}$$

$$\mathbf{z} = \begin{bmatrix} z_1 & z_2 & \vdots & z_m \end{bmatrix}_{1\times m}$$

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1m} \\ w_{21} & w_{22} & \cdots & w_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ w_{d1} & w_{m2} & \cdots & w_{dm} \end{bmatrix}_{d\times m} \qquad \mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}_{1\times d}$$

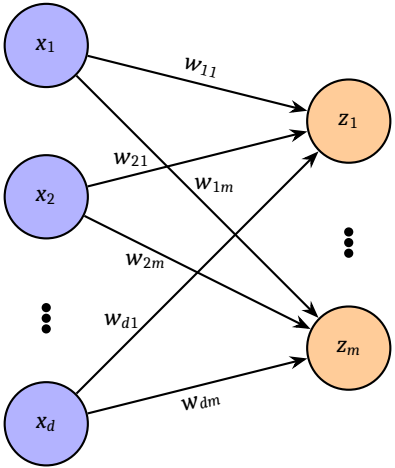
$$\Rightarrow \mathbf{z} = \mathbf{x}W$$

Linear Layer: Case of Hidden Layer



That was the case of one sample, if we consider n sample, then?

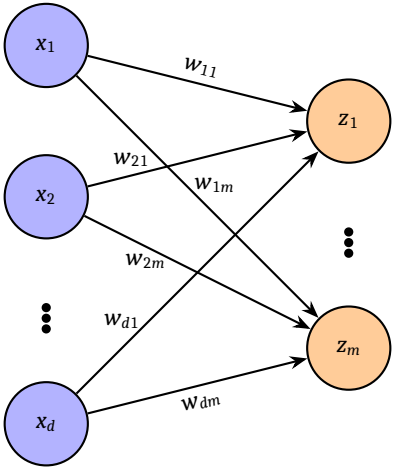
Linear Layer: Case of Hidden Layer



That was the case of one sample, if we consider n sample, then?

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

Linear Layer: Case of Hidden Layer



That was the case of one sample, if we consider n sample, then?

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

$$\Rightarrow Z = XW$$

Nonlinear Operation

$$A = \sigma(Z)$$

which will use elementwise operation.

Training a Neural Network

How to Train your Neural Network

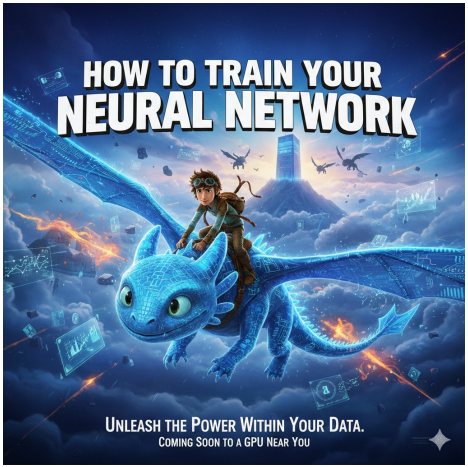


Image generation from Google Gemini.

Training a Neural Network

Some criteria to consider:

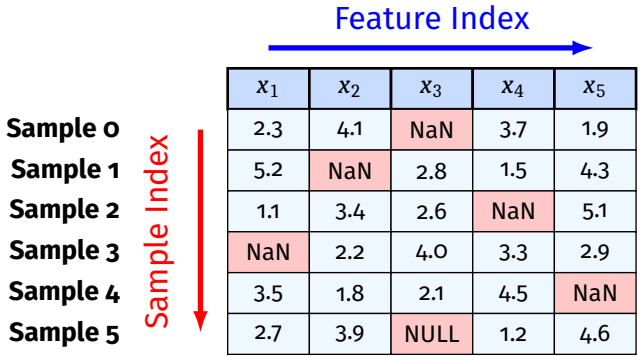
1. Data Preprocessing: Normalization/Standardization
2. Batch Size
3. Batch Normalization
4. Dropout
5. Early-Stopping
6. Learning-Rate Scheduling
7. Optimizer Selection
8. Weight Initialization
9. Activation Functions
10. Network Depth & Width
11. Skip/Residual Connections
12. Regularization
13. Loss Functions
14. Gradient Clipping
15. Warm-Up Scheduling

Data Preprocessing

Imputing Missing Values

Definition

The process of filling miss values or correct known errors is called **imputation**.



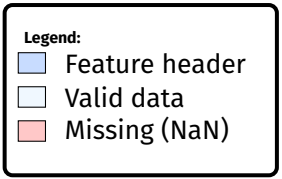


Figure: Data Table Structure: Features as Columns, Samples as Rows. Red cells indicate missing values (NaN) that require imputation before model training.

Common Types of Missing Values

1. Missing Completely at Random (MCAR), i.e. the probability of being missing is not related to the data and is only dependent on some parameter ϕ.
2. Missing at Random (MAR), i.e., the missingness probability is only related to the observed variables in the data.
3. Missing not at random (MNAR), neither of the above, and the missingness probability is related to missing values of the incomplete variable, which are unknown to us, even after conditioning on observed information

Common Imputation Methods

1. Get rid of those samples where missing # features exceed certain number (or percentage)
2. Get rid of feaures where # missing values exceed certai numbers (or percentage)
3. Sample Mean (only for continuous data)
4. Sample Median (only for continuous data)
5. k-Nearest Neighbors (kNN) imputation
6. Expectation-Maximization (EM) imputation
7. Bayesian Approach

Mean and median approach or anything similar would fail to quantify uncertainty in the estimated missing value.

Bayesian Estimation for Missing Data

1. Model your data first (probabilistic modeling): e.g. normal distribution.
2. Compute prior distribution over uknown parameters.
3. Calculate posterior: update the prior using observed data as (Bayes' Theorem in the play). This gives a posterior distribution over model parameters and missing values.
4. Draw samples using Markov Chain Monte Carlo simulation that will be plausible missing values.

Under a Bayesian framework, missing observations can be thought of as any other parameter in the model whose distribution needs to be determined.

Bayesian Framework

Posterior Distribution

$$p(\boldsymbol{\theta} \mid X) = \frac{p(\mathbf{X} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{\int_{-\infty}^{\infty} p(\mathbf{X} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}} \propto p(\mathbf{X} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})$$

$p(\mathbf{X} \mid \boldsymbol{\theta})$ is likelihood, $p(\boldsymbol{\theta})$ is prior.

$$p(\mathbf{X}_{mis} \mid \mathbf{X}_{obs}) = \int p(\mathbf{X}_{mis} \mid \mathbf{X}_{obs}, \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \mathbf{X}_{obs})\, d\boldsymbol{\theta}.$$

Calculating posterior is hard, so we use Monte Carlo methods (Markov Chain Monte Carlo (MCMC)).
An example code for MCMC imputation using PyMC is available at
https://www.pymc.io/projects/examples/en/latest/howto/Missing_Data_Imputation.html

Normalization/Standardization

Definition

In simpler terms: adjusting values measured on different scales to a common scale is **Normalization**.

Otherwise, aligning entire probability distribution of adjusted values is **Normalization**.

Standard Normalization

$$\hat{x} = \frac{x - \mu}{\sigma}$$

Minmax Normalization

$$\hat{x} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Impact of Batch Size in the Training Process

Batch Size

Definition

The number of samples passed through the neural network simultaneously is called **batch size**.

Determining a suitable batch size is an important consideration to the training process, as it may influence the learning rate of the model.

$$\mathbf{w} = \mathbf{w} - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla_{\mathbf{w}} \mathcal{L}_i(\mathbf{w})$$

m is the batch size.

How does the batch size affect training?

Stochastic Gradient Descent and Batch Size

❓ What's problem if we use entire training set for gradient descent?

Stochastic Gradient Descent and Batch Size

❓ What's problem if we use entire training set for gradient descent?

Stochastic gradient descent provides approximation of true gradients using one sample at a time after shuffling the training sample and pick one training sample to update the weights. It optionally adds a small noise to the gradient calculation.

Mini Batch

Single sample gradient calculation can be very slow, so we can choose more than one sample at a time (but not all of them). This is called **mini batch**.

Suggested Reading

1. Masters, Dominic, and Carlo Luschi. **"Revisiting small batch training for deep neural networks."** arXiv preprint arXiv:1804.07612 (2018).
2. You, Yang, Yuhui Wang, Huan Zhang, Zhao Zhang, James Demmel, and Cho-Jui Hsieh. **"The limit of the batch size."** arXiv preprint arXiv:2006.08517 (2020).
3. Smith, Samuel L., Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. **"Don't decay the learning rate, increase the batch size."** arXiv preprint arXiv:1711.00489 (2017).

Batch Normalization

Normalization applied per batch:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$\mu_{\mathcal{B}}$ batchwise mean, $\sigma_{\mathcal{B}}^2$ is batchwise variance, and ϵ is to avoid numerical instability.

Batch Normalization

But why Batch Normalization?

Batch Normalization

But why Batch Normalization?

1. Batch Normalization smoothened out the loss landscape.
2. Gradients become more reliable and predictive.

Reading:

1. Santurkar, Shibani, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. **"How does batch normalization help optimization?."** Advances in neural information processing systems 31 (2018).
2. Section 3.3 of Huang, Lei. Normalization Techniques in Deep Learning. Cham: Springer, 2022.

Preventing Overfitting and Regularization

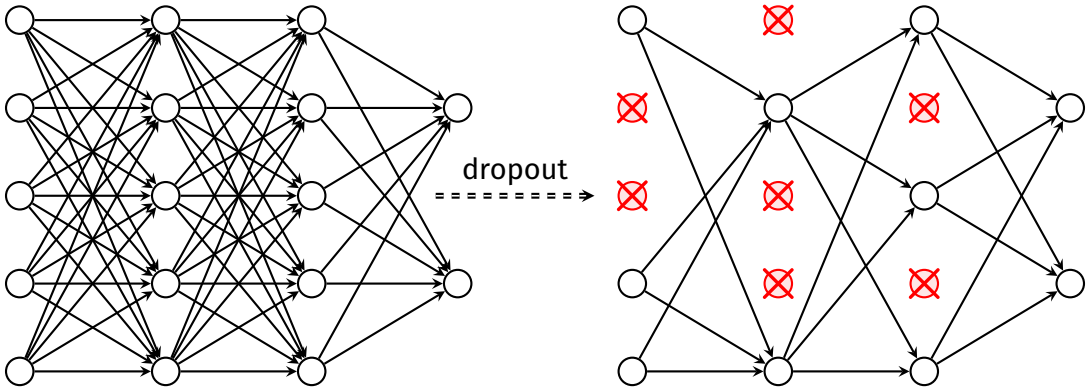
Dropout

Dropout: A new way to regularize a network

- The central idea behind *dropout* is to randomly dropout nodes in the neural network with a probability p. After the nodes are "dropped out" the remain network is a subnetwork of the original.
- Dropout has been shown to work quite well at preventing overfitting and allowing the network to find a good local minimum.
- **Recommendation**: Use dropout!

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

Dropout in a Neural Network



Dropout Model

During training, Dropout stochastically ”thins” the network by sampling a mask from a Bernoulli distribution.

$$
\begin{aligned}
r_i^{(l)} &\sim \text{Bernoulli}(p) && \text{(Mask)} \\
\tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} \otimes \mathbf{y}^{(l)} && \text{(Thinned Output)} \\
\mathbf{z}^{(l+1)} &= W^{(l+1)} \tilde{\mathbf{y}}^{(l)} + \mathbf{b}^{(l+1)} \\
\mathbf{y}^{(l+1)} &= \sigma(\mathbf{z}^{(l+1)})
\end{aligned}
$$

- $\mathbf{r}^{(l)}$ is a vector of independent Bernoulli random variables, each of which has probability p of being 1.
- The operator $\otimes$ denotes an element-wise product.
- σ is an activation function.
- The thinned outputs as input to the next layer.
- For learning, the derivatives of the loss function are backpropagated through the thinned network.
- At test time, the weights are scaled as $W_{test}^{(l)} = pW^{(l)}$. The resulting neural network is run without dropout.

Regularization in Neural Network

L2 Regularization

$$\hat{\mathcal{L}}(W) = \mathcal{L}(W) + \frac{\alpha}{2}||W||_2^2$$

L1 Regularization

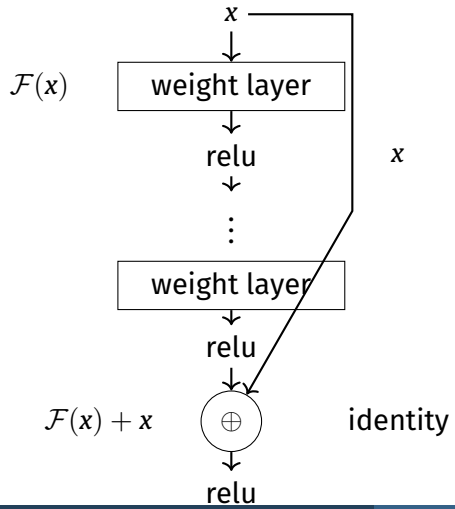
$$\hat{\mathcal{L}}(W) = \mathcal{L}(W) + \frac{\beta}{2}||W||_1$$

1. L1 regularization promotes sparsity by forcing weights to zero.
2. L2 regularization shrinks weights, but some of the components of weights large than others that are contributing more to the model.

Network Architecture Consideration

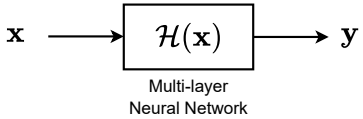
Skip/Residual Connection

A shallower layer directly connected to a much deeper layer in a neural network is called **Skip Connection** or **Residual Connection**.



Skip/Residual Connection

Prevents Vanishing or Exploding Gradients.
Consider a regular ANN:



If a multilayer nn can mdel $\mathcal{H}$, then it may also be able to model residual $\mathcal{H}(\mathbf{x}) - \mathbf{x}$, assuming input/ouput have same dimensions. So rather than expecting layers to model $\mathcal{H}$, we expect them to model $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$, and the using a skip connection, we have $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ later.

So input-ouput mapping becomes:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

Reference: He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.

Number of layers and neurons to consider

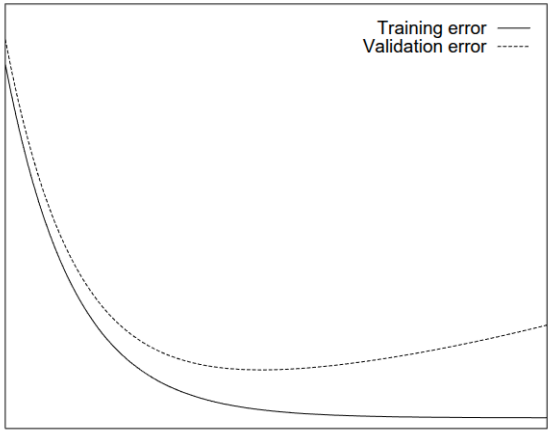
Consider following citieria:

1. How does the peformance change as we increase the number of layers?
2. Do training time scale with number of layers?
3. What happens to the training performance if you add more neurons?

Early Stopping

Stop the training process when some conditions are fulfilled to avoid overfitting.

Usually adhoc, but we can establish some rules. Use validation set for early stopping.



Early Stopping

$\mathcal{L}$ be the loss (or objective or cost) function. $\mathcal{L}_{tr}$ be the training loss, $\mathcal{L}_{val}$ be the validation loss. We can define the lowest validation set error obtained in epochs upto t epochs:

$$\mathcal{L}_{opt}(t) := \min_{t' \leq t} E_{val}(t')$$

We can define generalization loss as

$$G(t) = 100\left(\frac{\mathcal{L}_{val}(t)}{\mathcal{L}_{opt}(t)} - 1\right)$$

It means we define generalization loss at epoch t as relative increase of validation error over the minimum so far. And, we specify a stopping criteria as, **stop when generalization loss after epoch t exceeds a certain threshold α.**

But let it train for a while before early stopping

To avoid stopping while the model is still learning rapidly, we define **training progress** over a strip of length k:

$$P_k(t) := 1000 \cdot \left(\frac{\sum_{t'=t-k+1}^{t} E_{tr}(t')}{k \cdot \min_{t'=t-k+1}^{t} E_{tr}(t')} - 1 \right)$$

- Measures how much **average** error exceeds **minimum** error in a strip.
- High values indicate unstable or rapidly improving phases.
- Approaches zero as training stabilizes.

Stopping Criteria: Generalization vs. Progress

We can define a criterion using the quotient of generalization loss (G) and training progress (P_k):

PQ_α : stop at first end-of-strip epoch t where

$$\frac{G(t)}{P_k(t)} > \alpha$$

- **Logic:** Only stop when the ratio of "error increase" to "learning speed" is too high.
- Assumes strips of length $k = 5$ for standard measurements.

Stopping Criteria: Successive Increases

The **third class** of stopping criteria relies on the sign of changes in generalization error.

UP_s : stop if UP_{s-1} stops after epoch $t-k$ and
$E_{va}(t) > E_{va}(t-k)$

UP_1 : stop after first end-of-strip epoch t with $E_{va}(t) > E_{va}(t-k)$

- Training stops when generalization error increases in s successive strips.
- This assumes consecutive increases indicate the start of final overfitting.

Advantages of the UP Criteria

- **Magnitude Independent**: The decision is independent of how large the increases actually are.
- **Local Measurement**: Changes are measured locally rather than against global minima.
- **Application**: Useful in pruning algorithms where errors may remain high over long periods.

But remember, these is just one kind of early stopping example, you can design your own.
Reference: Prechelt, Lutz. ”Early stopping-but when?.” In Neural Networks: Tricks of the trade, pp. 55-69. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002.

References and Additional Reading

1. He, Yulei, Guangyu Zhang, and Chiu-Hsieh Hsu. **Multiple imputation of missing data in practice: basic theory and analysis strategies**. Chapman and Hall/CRC, 2021.
2. Huang, Lei. Normalization Techniques in Deep Learning. Cham: Springer, 2022.

Learning Rate Scheduling

Automatically adjust the learning rate during training.

1. **StepLR:** reduces the learning rate by a fixed factor at regular intervals.
2. **ExponentialLR:** reduces the learning rate by multiplying it by a decay factor each epoch.
3. **CosineAnnealingLR:** follows a cosine curve, starting high and smoothly decreasing to a minimum value.
4. **ReduceLROnPlateau:** monitoring validation metrics and reducing the learning rate only when improvement stops happening.

Warm-Up Scheduling

Warmup scheduling is a strategy used in deep learning to stabilize the training of complex neural network architectures. It addresses the trade-off between optimization stability and training speed.

1. For simplicity, the learning rate typically increases linearly during the warmup period until it reaches a specified initial maximum.
2. Once the warmup is complete, the learning rate is gradually decreased (cooled down) until the optimization process concludes.

Optimizers

Different Kind of Optimizers

Gradient Descent is one of the many optimizers that can be used to optimize a function and find its minima. Some other commonly popular optimizers are

1. RMSProp
2. Adagrad (Adaptive Gradient)
3. SGD with Momentum
4. SGD with Nesterov Accelerated Gradient
5. AdaDelta
6. ADAM

RMSProp

RMSProp (Root Mean Square Propagation) is an adaptive learning rate method that divides the learning rate by an exponentially decaying average of squared gradients.

- Update rule:

$$\begin{aligned} g_t &= \nabla_\theta J(\theta_t) \\ v_t &= \beta v_{t-1} + (1-\beta) g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t \end{aligned} \tag{1}$$

- η: Learning rate
- β: Decay rate (typically 0.9)
- ϵ: Small constant for numerical stability

Adagrad (Adaptive Gradient)

Adagrad adapts the learning rate to the parameters, performing smaller updates for frequently occurring features and larger updates for infrequent ones.

- Update rule:

$$\begin{aligned} g_t &= \nabla_\theta J(\theta_t) \\ G_t &= G_{t-1} + g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t \end{aligned} \tag{2}$$

- η: Learning rate
- G_t: Accumulated squared gradients
- ϵ: Small constant for numerical stability

SGD with Momentum

SGD with Momentum accelerates gradient descent by adding a fraction of the previous update to the current update.

- Update rule:

$$\begin{aligned} g_t &= \nabla_\theta J(\theta_t) \\ v_t &= \gamma v_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - v_t \end{aligned} \tag{3}$$

- η: Learning rate
- γ: Momentum term (typically 0.9)
- v_t: Velocity vector

SGD with Nesterov Accelerated Gradient

Nesterov Accelerated Gradient (NAG) improves momentum by calculating the gradient at the estimated future position.

- Update rule:

$$\begin{aligned} g_t &= \nabla_\theta J(\theta_t - \gamma v_{t-1}) \\ v_t &= \gamma v_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - v_t \end{aligned} \tag{4}$$

- η: Learning rate
- γ: Momentum term (typically 0.9)
- v_t: Velocity vector

AdaDelta

AdaDelta is an extension of Adagrad that reduces its aggressive, monotonically decreasing learning rate.

- Update rule:

$$
\begin{aligned}
g_t &= \nabla_\theta J(\theta_t) \\
E[g^2]_t &= \rho E[g^2]_{t-1} + (1-\rho) g_t^2 \\
\Delta\theta_t &= -\frac{\sqrt{E[\Delta\theta^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} g_t \\
E[\Delta\theta^2]_t &= \rho E[\Delta\theta^2]_{t-1} + (1-\rho)\Delta\theta_t^2 \\
\theta_{t+1} &= \theta_t + \Delta\theta_t
\end{aligned}
\tag{5}
$$

- ρ: Decay rate (typically 0.9)
- ϵ: Small constant for numerical stability

ADAM (Adaptive Moment Estimation)

ADAM combines the benefits of RMSProp and Momentum by adapting learning rates and using momentum.

- Update rule:

$$
\begin{aligned}
g_t &= \nabla_\theta J(\theta_t) \\
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\
\hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\
\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t
\end{aligned}
\tag{6}
$$

- η: Learning rate
- β_1, β_2: Exponential decay rates (typically 0.9 and 0.999)
- ϵ: Small constant for numerical stability

Stabilizing Gradients: Value Clipping

In **value clipping**, each element of the gradient vector g is independently forced into the range $[-c, c]$:

$$g_i \leftarrow \max(-c, \min(c, g_i))$$

- **Hard Thresholding**: Prevents any single weight update from being inappropriately large.
- **Effect on Direction**: Unlike norm clipping, this may change the *direction* of the gradient vector.
- **Implementation**: Often used to prevent "jitter" in unstable phases of training.

Stabilizing Gradients: Norm Clipping

To prevent exploding gradients in unstable optimization problems, we rescale the gradient vector g if its norm exceeds a threshold τ:

$$g \leftarrow \min\left(1, \frac{\tau}{\|g\|}\right) g$$

- **Direction Preservation**: Unlike value clipping, norm clipping maintains the direction of the update.
- **Synergy with Warmup**: Both techniques mitigate divergence in advanced network designs.

Customization of Neural Network Architecture

`nn.Module` in PyTorch

`nn.Module`: The fundamental building block of PyTorch neural networks.

Basic structure

1. Abstract class
2. A layer or collection of layers
3. Encapsulates state (parameters) and behavior (operations).
4. Can contain another `nn.Module` object.

Everything is a Module

PyTorch follows the principle that neural networks should be composable from simple, reusable components. Whether it's a single linear layer or a 1000-layer ResNet, both are `nn.Module` instances.

Philosophy Behind `nn.Module`

Design Principles

- **Everything is a Module**: From single layer to 1000-layer ResNet
- **Separation of Definition and Execution**:

$$\text{Definition} \neq \text{Execution}$$
$$\text{Graph structure} \neq \text{Forward pass}$$

- **Explicit Over Implicit**: Manual forward pass definition
- **Pythonic Design**: Uses standard Python inheritance and conventions
- **Minimal Abstraction**: Can drop to raw tensor operations when needed

$$\texttt{nn.Module} = \text{Container} + \text{Behavior} + \text{State}$$
$$\text{Parameters} \in \texttt{nn.Parameter} \subset \text{Tensor}$$
$$\text{Sub-modules} \in \texttt{nn.Module}$$

Philosophy Behind `nn.Module`

1. **Modularity:** Each nn.Module can be a small, reusable component. Complex networks are built by composing these modules.
2. **Encapsulation:** nn.Module encapsulates both the parameters (as nn.Parameter) and the operations (in the forward method). It also manages the state of the module, including training mode (train/eval) and device placement (CPU/GPU).
3. **Automatic Differentiation:** By defining the forward pass, the backward pass (gradients) is automatically handled by PyTorch's autograd engine, provided that the operations used in the forward pass are differentiable and track gradients.
4. **Hierarchical Structure:** nn.Module can contain other nn.Module instances, allowing for tree-like structures. This is how complex networks are built from simpler ones.
5. **Parameter Management:** nn.Module automatically tracks all nn.Parameter objects that are assigned as attributes. It also provides methods for moving parameters to different devices, saving and loading, etc.
6. **Hooks:** nn.Module allows for the registration of hooks, which are functions that can be called at various stages (before/after forward/backward). This is useful for debugging, visualization, and extending functionality.

A Class Diagram

```
nn.Module
|--- attributes:
|   |--- _parameters: Dict[str, Parameter]
|   |--- _modules: Dict[str, Module]
|   |--- _buffers: Dict[str, Tensor]
|   |--- training: bool
|--- methods:
|   |--- __init__()
|   |--- __setattr__(name, value)
|   |--- parameters() -> Iterator[Parameter]
|   |--- named_parameters() -> Iterator[(str, Parameter)]
|   |--- modules() -> Iterator[Module]
|   |--- named_modules() -> Iterator[(str, Module)]
|   |--- forward(*args, **kwargs) -> Tensor
|   |--- train(mode: bool)
|   |--- eval()
|   |--- to(device: Device)
|   |--- state_dict() -> Dict[str, Tensor]
|   |--- load_state_dict(state_dict: Dict[str, Tensor])
|--- abstract methods:
|   |--- forward(*args, **kwargs) -> Tensor (to be implemented by subclasses)
```

Forward Pass

Forward is defined in `forward` function of `nn.Module` that has to be implemented for forward propagation structure.
A very simplified view:

```
class Module:
    def __init__(self):
        self._parameters = {}        # Trainable parameters
        self._buffers = {}           # Non-trainable state
        self._modules = {}           # Child modules
        self.training = True         # Training mode flag
        self._forward_hooks = None   # Hooks for customization
        self._backward_hooks = None
    def forward(self, *input):
        """Defines the computation performed at every call.
        Should be overridden by all subclasses.
        """
        raise NotImplementedError
    def __call__(self, *input, **kwargs):
        """Enables calling the module like a function.
        This is where everything happens.
        """
        # Set up hooks, handle training mode, etc.
        result = self.forward(*input, **kwargs)
        # Clean up hooks, etc.
        return result
```

Parameters in `nn.Module`

The workflow of creating- a Neural Network object for training must register parameters that need to be optimized.

Parameter registration answers the question: **which parameters to optimize**

as, there can be more attributes in a nn.Module class that won't be to optimized.

```
self.weight = nn.Parameter(torch.Tensor(out_features, in_features))
```

Other things happen under the hood on registering a parameter:

1. Only registered parameters are saved/loaded
2. Registered tensors move together with `.to(device)`
3. Only registered parameters get gradients

How Parameter Registration Works Internally

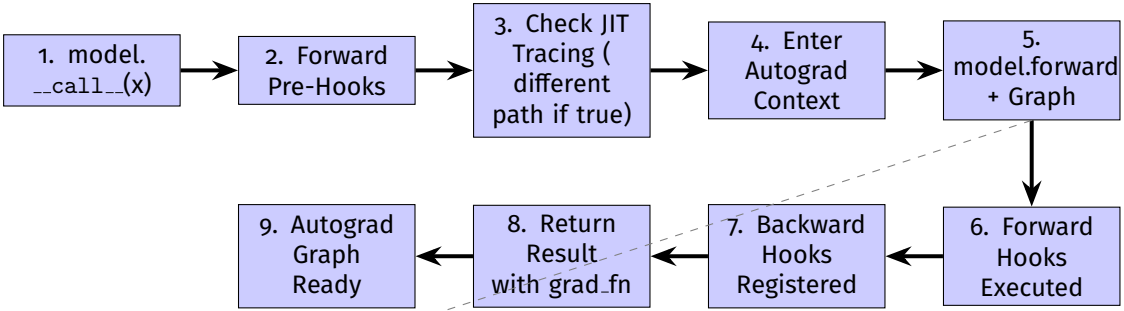
Using `__setattr__`

```python
class MyModule:
    def __setattr__(self, name, value):
        # Special handling for Parameter, Module, Buffer
        if isinstance(value, nn.Parameter):
            self._parameters[name] = value
        elif isinstance(value, nn.Module):
            self._modules[name] = value
        elif isinstance(value, torch.Tensor):
            if name in self._buffers:
                self._buffers[name] = value
        # Call parent for normal attributes
        super().__setattr__(name, value)
```

Parameter Types

Parameters	→ Learned via gradient descent
Buffers	→ Saved but not learned

What REALLY happens when you do `model(x)`?



1. Submodules recursively call
2. `__call__`, operations create Function nodes
3. Builds computation graph

Layers in Neural Network Construction

Inherits `nn.Module`

Layers can be treated as composable, independent building blocks

Some Custom Layers for Feature Engineering

```python
class SquareLayer(nn.Module):
    """x -> x^2"""
    def forward(self, x):
        return x ** 2

class PolynomialLayer(nn.Module):
    def __init__(self):
        super().__init__()
        self.a = nn.Parameter(torch.tensor(1.0))
        self.b = nn.Parameter(torch.tensor(0.0))
        self.c = nn.Parameter(torch.tensor(0.0))

    def forward(self, x):
        return self.a * x**2 + self.b * x + self.c
```

```python
class InteractionLayer(nn.Module):
    """Creates interaction features"""
    def forward(self, x):
        batch_size, n_features = x.shape
        interactions = []
        for i in range(n_features):
            for j in range(i, n_features):
                interactions.append(
                    x[:, i:i+1] * x[:, j:j+1]
                )
        return torch.cat([x] + interactions, dim=1)
```

Some Custom Layers for Feature Engineering

```python
class ZScoreNormalization(nn.Module):
    """Normalizes input to zero mean and unit variance along
    ↪ specified dimension"""
    def __init__(self, dim=-1, eps=1e-5):
        super().__init__()
        self.dim = dim
        self.eps = eps

    def forward(self, x):
        mean = x.mean(dim=self.dim, keepdim=True)
        std = x.std(dim=self.dim, keepdim=True)
        return (x - mean) / (std + self.eps)
```

```python
class LogTransform(nn.Module):
    """Applies log transformation with offset for positive values:
    ↪ log(x + offset)"""
    def __init__(self, offset=1.0):
        super().__init__()
        self.offset = offset

    def forward(self, x):
        return torch.log(x + self.offset)
```

Module Hierarchy with Layers within a Module

```python
class Network(nn.Module):
    def __init__(self):
        super().__init__()
        # Child modules are automatically registered
        self.layer1 = nn.Linear(10, 20)
        self.layer2 = nn.Linear(20, 5)

    def forward(self, x):
        return self.layer2(self.layer1(x))

# Accessing the hierarchy:
model = Network()
print(list(model.children()))  # [layer1, layer2]
print(list(model.parameters()))  # All parameters from both layers
```

Hooks in `nn.Module`

What is a Hook?

In PyTorch, a hook is simply a function that you attach to either a Tensor or a `nn.Module`, and it gets executed automatically when the forward (not the same as forward function, but literally forward operation in the computation graph) or backward pass happens.

1. Forward Hooks: Executed during forward pass
2. Backward Hooks: Executed during backward pass

A hook is a function with a specific signature. When we say a hook is executed, we are talking about this function being executed. Such functions are called **callbacks**.

They let us inspect, modify, or intercept the flow of data in our neural network without changing its source code.

Forward and Backward Hooks

Forward hook signature:

```
hook(module, input, output) -> None
```

Backward hook signature:

```
hook(module, grad_input, grad_output) -> Tensor or None
```

Forward Hook Example

```python
import torch
import torch.nn as nn

# Define a simple model
model = nn.Linear(5, 3)

# Define a hook function
def print_output(module, input, output):
    print(f"Output: {output}")

# Register the hook
handle = model.register_forward_hook(print_output)

# Run the model
x = torch.randn(1, 5)
output = model(x)

# Remove the hook
handle.remove()
```

Forward hooks can't modify the gradient.

Backward Hook Example

```python
import torch
import torch.nn as nn

# Define a simple model
model = nn.Linear(5, 3)

# Define a hook function
def print_output(module, input, output):
    print(f"Output: {output}")

# Register the hook
handle = model.register_forward_hook(print_output)

# Run the model
x = torch.randn(1, 5)
output = model(x)

# Remove the hook
handle.remove()
```

1. Can access input, output, and gradients of the module
2. Can modify the gradient (be careful with this one!)

The End